Création d'interfaces graphiques avec Python & Qt 1
I-Introduction 1
II- Comparatif des différents outils de création d'interface graphique sous Python 1
III- Bibliothèque Qt 1
1- Pourquoi Qt ? 1
2- Présentation 1
3- Outil 1
4- Installation 2
5- Conception de l'interface graphique avec Qt Designer 2
6- Utilisation du fichier de l'interface graphique 8
7- Connexions des signaux aux slots 9
Ateliers 10
Atelier N°1 : Création d'une calculatrice 11
Atelier N°2 : Création d'un générateur de mot de passe 13
Atelier N°3 : Création d'un Gestionnaire des contacts passes 15
Atelier N°4 : Cryptage des mots de 17
Annexe 19

Création d'interfaces graphiques avec Python & Qt

I-Introduction :

Les interfaces graphiques (ou interfaces homme-machine) sont appelées GUI (pour Graphical User Interface). Elles permettent à l'utilisateur d'interagir avec un programme informatique, grâce à différents objets graphiques (boutons, menus, cases à cocher…). Ces objets sont généralement actionnés à l'aide de la souris ou du clavier.

Malgré le fait que les interfaces graphiques semblent secondaires par rapport au développement du coeurd'une application, elles doivent néanmoins être conçues et développées avec soin et rigueur.

Leur efficacité et leur ergonomie sont essentielles dans l'acceptation et l'utilisation de ces outils par les utilisateurs finaux.

Une bonne conception et un développement maitrisé permettent également d'en assurer une meilleure maintenabilité.

Une interface graphique est constituée de **composants** (ou encore **widgets**) qui sont la source **d'évènements**.

- Composants ou widgets : ce sont des boutons, zones de saisie, zones de dessin, labels, listes déroulantes, boîtes à cocher, bouton radio …
- Un évènement : clic sur un bouton de la souris, saisie d'un texte, toucher Entrée, flèches du clavier …

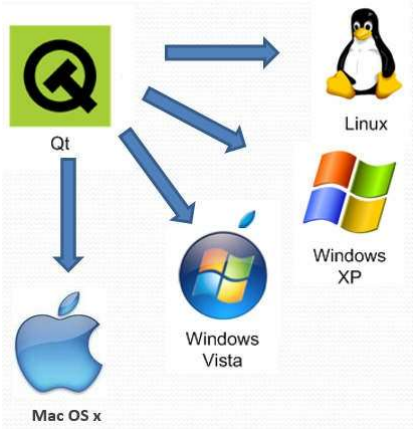
II-Comparatif des différents outils de création d'interface graphique sous Python :

Bibliothèque	Plateforme	Languages supportés
Qt5	Windows, Mac, GNU/Linux	C++ , **Python** , Java, Ruby, C#
FLTK	Windows, Mac, GNU/Linux	C++ , Ruby, **Python**
FOX	Windows, Mac, GNU/Linux	C++ , **Python** , Ruby3
GTK+	Windows, Mac, GNU/Linux	C++ ,C# , Java , **Python** , Ruby
Tkinter	Windows, Mac, GNU/Linux	**Python**
WideStudio	Windows, Mac, GNU/Linux	C++ , Java, Perl, Ruby, **Python**
wxWidgets	Windows, Mac, GNU/Linux	C++ , **Python** , perl, java, C#

III-Bibliothèque Qt :

1- Pourquoi Qt ?

- ✓ Performance (C++)
- ✓ Relativement Simple
- ✓ Gratuit (GPL) et code source
- ✓ Nombreux outils
  - ➢ Générateur d'interface : Qt Designer
  - ➢ Internationalisation : Qt Linguist
  - ➢ Documentation : Qt Assistant
  - ➢ Exemples : Qt Examples
  - ➢ Programmation : Qt Creator (eclipse)
- ✓ Multi-Plateformes
  - ➢ Look and feel simulé



2- Présentation :

PyQt est la contraction de deux mots : d'un côté, Python (le langage de programmation utilisé) réputé fort simple d'apprentissage ; de l'autre, Qt, un outil extrêmement complet (principalement pour des interfaces graphiques). PyQt sert de couche de liaison entre ces deux mondes et apporte Qt à l'environnement Python.

3- Outil :

Qt Designer est un outil permettant de créer rapidement des interfaces utilisateur graphiques avec des widgets du framework Qt GUI. Il vous offre une interface simple de glisser-déposer pour la mise en page de composants tels que des boutons, des champs de texte, des zones de liste déroulante, etc.

Qt Designer produit des fichiers **.ui**.

Il s'agit d'un format XML spécial qui stocke vos widgets sous forme d'arborescence. Vous pouvez soit charger ces fichiers au moment de l'exécution, soit les faire traduire dans un langage de programmation tel que C ++ ou Python (à l'aide de l'utilitaire pyuic).

4- Installation :

Qt en Python s'installe via un package appelé **PyQt**. PyQt contient toute l'API nécessaire pour commencer à créer des interfaces graphiques.

- Pour installer Qt5 tapez :

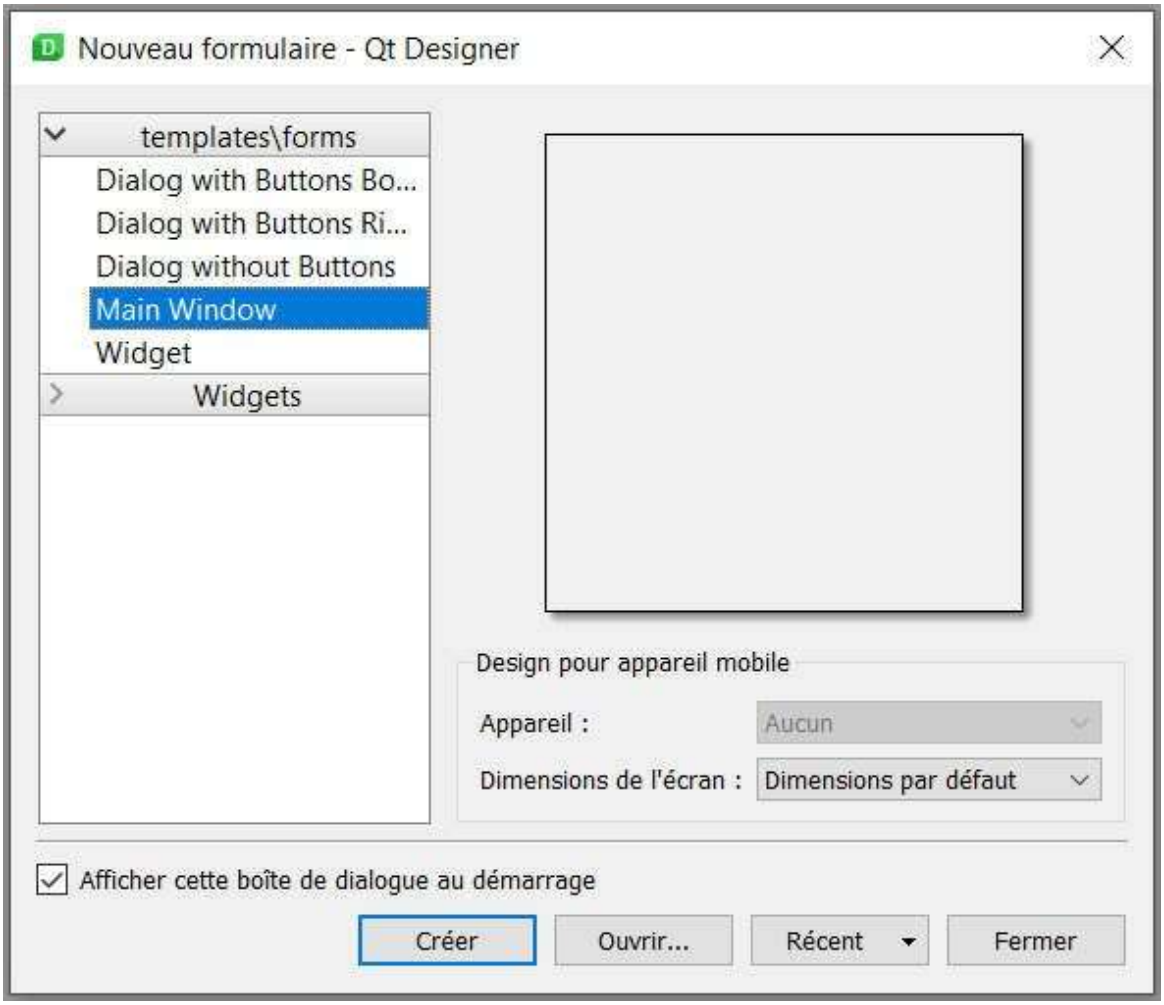
  pip install pyqt5

- Et pour installer PyQt5 Designer

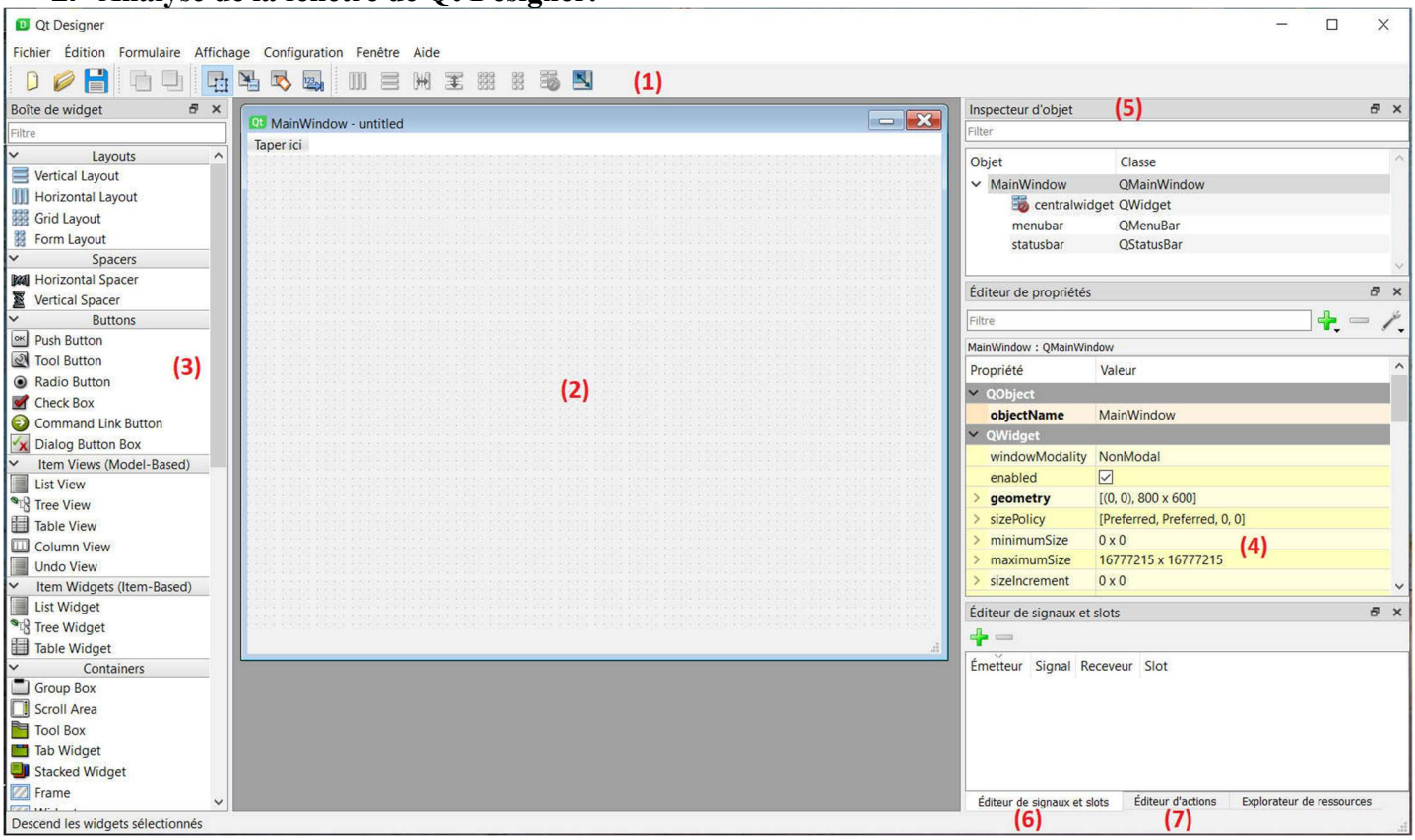
  pip install pyqt5-tools

5- Conception de l'interface graphique avec Qt Designer :

1. Choix du type de fenêtre à créer :



2. Analyse de la fenêtre de Qt Designer.



(1) Pour passer d'un mode d'édition à un autre. Qt Designer propose 4 modes d'édition :

- Edit Widgets : le mode par défaut, que vous utiliserez le plus souvent. Il permet d'insérer des widgets sur la fenêtre et de modifier leurs propriétés.
- Edit Signals/Slots : permet de créer des connexions entre les signaux et les slots de vos widgets.
- Edit Buddies : permet d'associer des **QLabel** avec leurs champs respectifs. Lorsque vous faites un layout de type **QFormLayout**, ces associations sont automatiquement créées.
- Edit Tab Order : permet de modifier l'ordre de tabulation entre les champs de la fenêtre, pour ceux qui naviguent au clavier et passent d'un champ à l'autre en appuyant sur la touche "Tab".

(2) Votre fenêtre (vide).

- Si vous créez une **QMainWindow**, vous aurez en plus une barre de menus et une barre d'outils.
- Si vous créez une **QDialog**, vous aurez probablement des boutons "OK" et "Annuler" déjà disposés.

(3) Widget Box : les widgets à placer sur la fenêtre.

(4) Property Editor : pour modifier toutes les propriétés du widget (y compris celles de ses ancêtres). Ce sont les "properties" de la doc **Qt5**

(5) Object Inspector : liste les widgets placés sur la fenêtre, sous forme d'arbre, chacun dans son conteneur.

(6) Signal / slot editor : pour connecter les signaux prédéfinis aux slots prédéfinis.

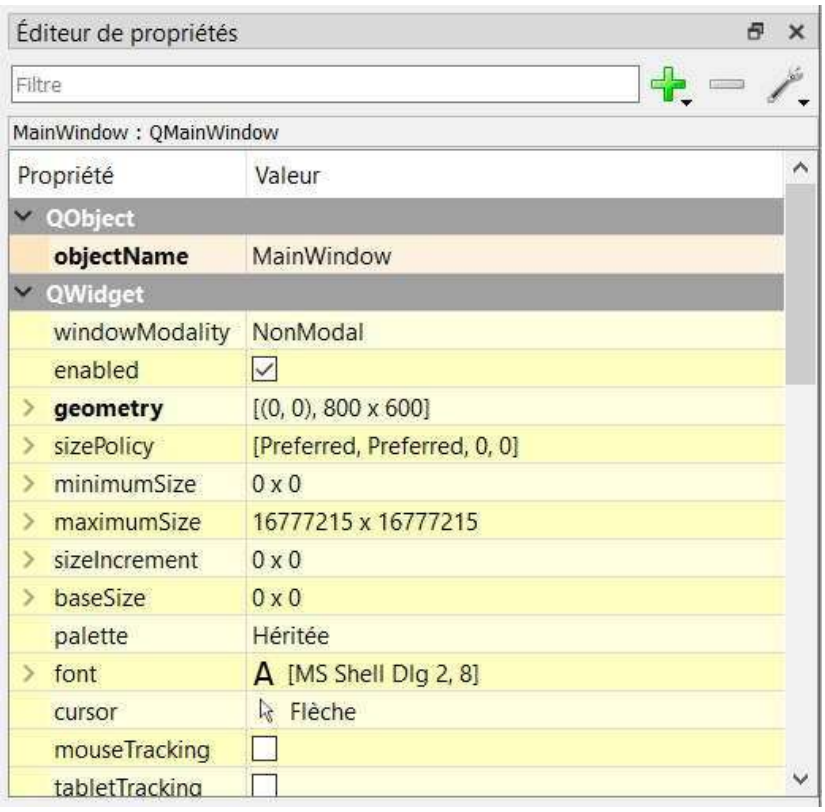
(7) Action Editor : permet de créer des **QAction** (pour les menus et barres d'outils).

2-1- Les layouts.

Les layouts **Qt5** sont conteneurs de widgets spécialisés dans le placement relatif des widgets, le positionnement sous forme de grille.

Toutes les classes de **layout** héritent de la classe de base abstraite **QLayout**.

2-2- Editeur des propriétés des widgets.

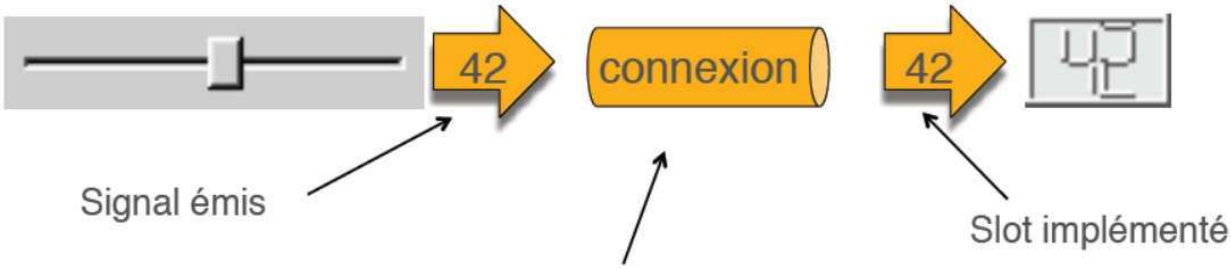


2-3-L'éditeur Signal-Slot

Contrairement à une application en mode console, qui est exécutée de manière séquentielle, une application basée sur l'interface graphique est pilotée par les événements. Les fonctions ou méthodes sont exécutées en réponse aux actions de l'utilisateur comme cliquer sur un bouton, sélectionner un élément d'une collection ou un clic de souris, etc., appelées événements.

Les widgets utilisés pour créer l'interface GUI agissent comme la source de ces événements. Chaque widget PyQt, qui est dérivé de la classe QObject, est conçu pour émettre un «signal» en réponse à un ou plusieurs événements. Le signal seul n'effectue aucune action. Au lieu de cela, il est «connecté» à un «slot». Le slot peut être n'importe quelle fonction Python appelable.

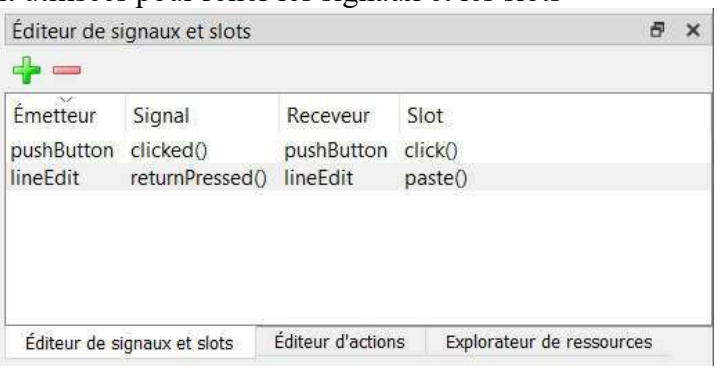
Dans PyQt, la connexion entre un signal et un slot peut être réalisée de différentes manières. Voici les techniques les plus couramment utilisées :



widget.signal.connect(slot_function)

Chaque objet PyQt dérivant de QObject prend en charge le mécanisme S&S

- Les widgets émettent des signaux
- Un signal annonce des changements d'état :
  - ✓ un bouton a été cliqué
  - ✓ une case à cocher est cochée / décochée
  - ✓ édition dans un champ de texte terminée
- Les widgets réagissent à un signal via les emplacements
- Les connexions sont utilisées pour relier les signaux et les slots



3. Hiérarchie des classes de l'API PyQt5.

L'API PyQt est une grande collection de classes et de méthodes. Ces classes sont définies dans plus de 20 modules. Voici quelques-uns des modules fréquemment utilisés :

N °	Modules et description
1	**QtCore** Classes non-GUI de base utilisées par d'autres modules
2	**QtGui** Composants de l'interface utilisateur graphique
3	**QtMultimedia** Cours de programmation multimédia de bas niveau
4	**QtNetwork** Cours de programmation réseau
5	**QtOpenGL** Classes de support OpenGL
6	**QtScript** Classes pour évaluer les scripts Qt
7	**QtSql** Classes pour l'intégration de base de données à l'aide de SQL
8	**QtSvg** Classes d'affichage du contenu des fichiers SVG

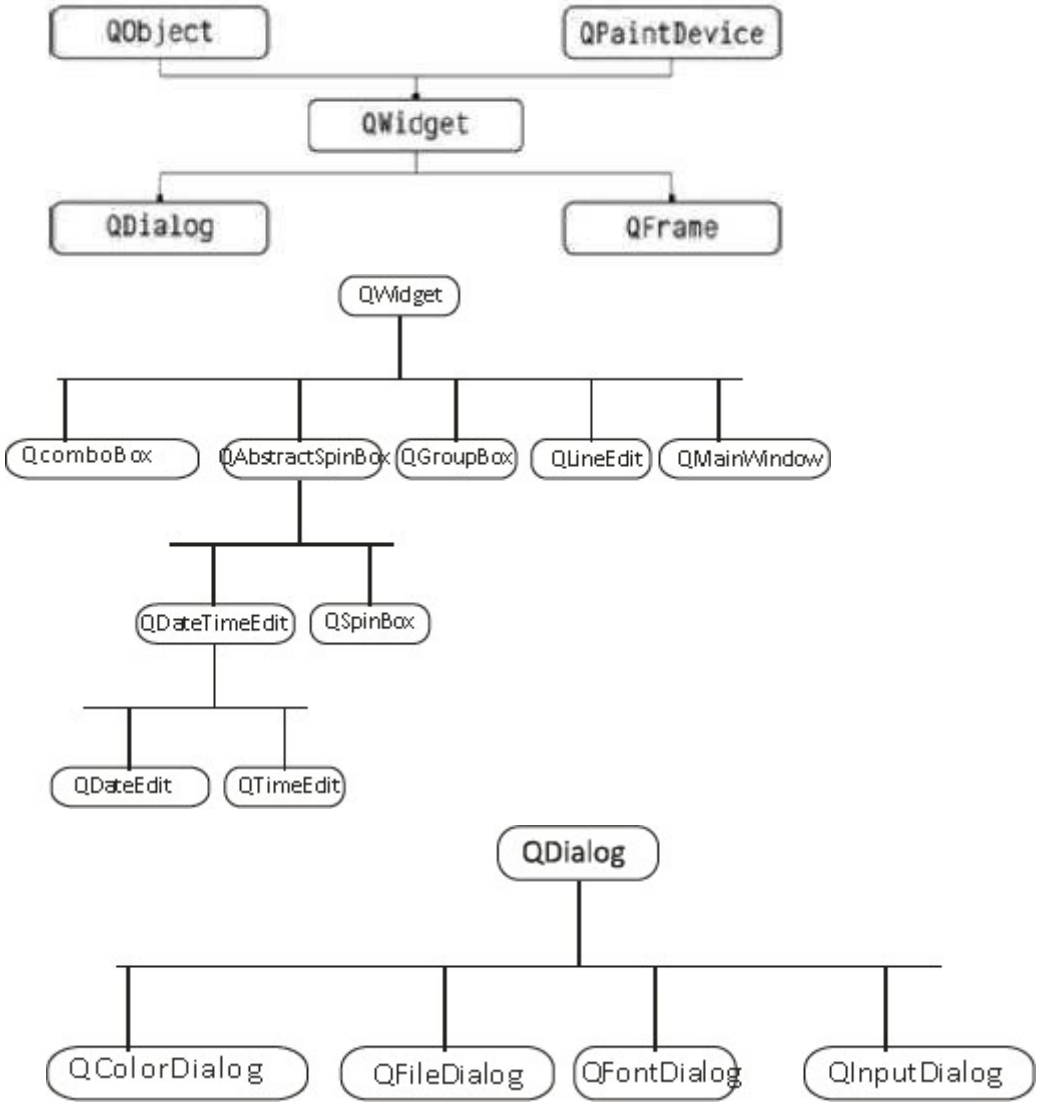
9	**QtWebKit** Classes de rendu et d'édition HTML
dix	**QtXml** Classes de gestion du XML
11	**QtAssistant** Assistance pour l'aide en ligne
12	**QtDesigner** Classes pour étendre Qt Designer

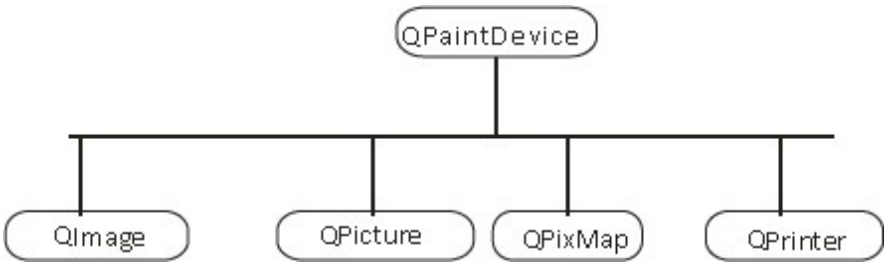
L'API PyQt contient plus de 400 classes. La classe **QObject** est au sommet de la hiérarchie des classes. C'est la classe de base de tous les objets **Qt**. En outre, la classe **QPaintDevice** est la classe de base pour tous les objets qui peuvent être peints.

La classe **QApplication** gère les principaux paramètres et le flux de contrôle d'une application GUI. Il contient une boucle d'événements principale à l'intérieur de laquelle les événements générés par des éléments de fenêtre et d'autres sources sont traités et distribués. Il gère également les paramètres à l'échelle du système et de l'application.

La classe **QWidget** , dérivée des classes **QObject** et **QPaintDevice** est la classe de base pour tous les objets d'interface utilisateur. Les classes **QDialog** et **QFrame** sont également dérivées de la classe **QWidget**. Ils ont leur propre système de sous-classes.

Les diagrammes suivants décrivent certaines classes importantes de leur hiérarchie.




QPaintDevice
QImage
QPicture
QPixMap
QPrinter

Vous trouverez ci-dessous les widgets couramment utilisés :

N °	Icone	Widgets et description
1	QLabel Name:	**QLabel** Utilisé pour afficher du texte ou une image
2	Peter	**QLineEdit** Permet à l'utilisateur de saisir une ligne de texte
3		**QTextEdit** Permet à l'utilisateur de saisir du texte sur plusieurs lignes
4	Click Me	**QPushButton** Un bouton de commande pour appeler une action
5	Name Address Phone number Mobile number	**QRadioButton** Permet d'en choisir une parmi plusieurs options
6	Checkbox 1 Checkbox 2 Tri-state button	**QCheckBox** Permet le choix de plusieurs options
7	10	**QSpinBox** Permet d'augmenter / diminuer une valeur entière
8		**QScrollBar** Permet d'accéder au contenu d'un widget au-delà de l'ouverture d'affichage
9		**QSlider** Permet de modifier la valeur liée de manière linéaire.
10	Python Python Java C++	**QComboBox** Fournit une liste déroulante d'éléments à sélectionner
11	Another File New Open Quit Ctrl+Q	**QMenuBar** Barre horizontale contenant des objets QMenu
12	Status Label	**QStatusBar** Habituellement, au bas de QMainWindow, fournit des informations d'état.
13	QToolBar	**QToolBar** Habituellement en haut de QMainWindow ou flottant. Contient des boutons d'action
14	Row1 Row2 Row3 Row4 Row5 Row6 Row7 Row8 Row9 Row10	**QListView** Fournit une liste sélectionnable d'éléments dans ListMode ou IconMode
15	Image	**QPixmap** Représentation d'image hors écran pour affichage sur un objet QLabel ou QPushButton
16	Text Input Dialog Enter your name: OK Cancel	**QDialog** Fenêtre modale ou non modale qui peut renvoyer des informations à la fenêtre parent

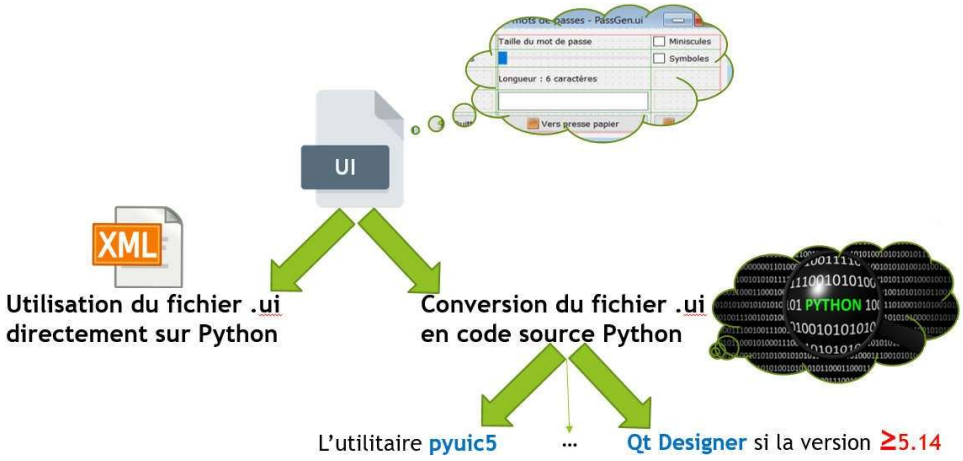
La fenêtre de niveau supérieur d'une application basée sur une interface graphique typique est créée parl'objet widget **QMainWindow**. Certains widgets énumérés ci-dessus prennent leur place dans cette fenêtre

principale, tandis que d'autres sont placés dans la zone centrale des widgets à l'aide de divers gestionnairesde disposition.

Le diagramme suivant montre le framework **QMainWindow**

6- Utilisation du fichier de l'interface graphique :

Pour exploiter l'interface graphique récemment crée avec **QtDesigner**, on peut soit la convertir en uneclasse en Python ou soit appeler directement le fichier sauvegardé par **QtDesigner** (**.ui**) depuis le programme Python.



▶ Conversion du fichier **.ui** en code Python à travers l'utilitaire **pyuic5**

```
Invite de commandes
Microsoft Windows [version 10.0.18363.1139]
(c) 2019 Microsoft Corporation. Tous droits réservés.

C:\Projet Générateur

C:\Projet Générateur>pyuic5 PassGen.ui -x -o PassGen.py
```

▶ Appel du fichier .ui en Python

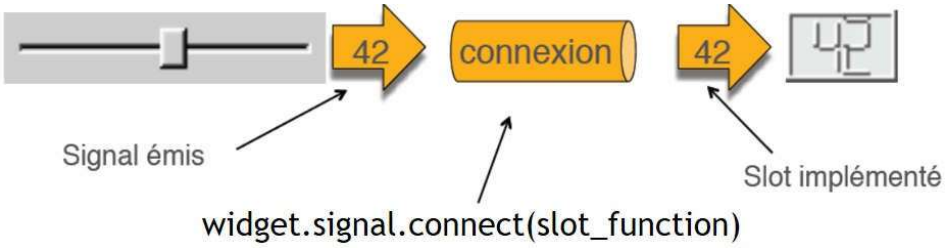
Pour charger des fichiers **.ui**, nous pouvons utiliser le module **uic** inclus avec **PyQt5**, en particulier la méthode **uic.loadUI ()**. Cela prend comme paramètre, le nom de fichier d'interface utilisateur (**.ui**) et le charge en créant un objet PyQt5 entièrement fonctionnel.

Exemple : ui=uic.loadUI("fichier.ui")

7-Connexions des signaux aux slots :

Si la définition des signaux/slots a été réalisée dans le fichier **.ui** (voir paragraphe précédent ***2-3-L'éditeur Signal-Slot***), ça ne reste qu'à définir des fonctions pour les slots portant les mêmes noms que précédemment. Sinon il faut commencer à créer les signaux provenant des widgets et les connecter aux slots puis définir les fonctions nécessaires pour ces derniers.

Syntaxe :



Exemple :

```
Import sys
from PyQt5 import uic
from PyQt5.QtWidgets import Qapplication
def on_generer():
        print("Bonjour")
app = QApplication(sys.argv)
ui = uic.loadUi("Exemple.ui")
ui.show()
ui.Generer.clicked.connect(on_generer)
```

Définition de la fonction à appeler (slot)

Composant (Widget)

Appel de la fonction du slot

Objet (fenêtre)

Ateliers

Widget QLabel

Un objet **QLabel** agit comme un espace réservé pour afficher du texte ou une image non modifiable, ou un film de GIF animé. Il peut également être utilisé comme clé mnémotechnique pour d'autres widgets. Du texte brut, un lien hypertexte ou un texte enrichi peuvent être affichés sur l'étiquette.
Le tableau suivant répertorie les méthodes importantes définies dans la classe **QLabel** :

N°	Méthodes et description
1	**setAlignment()** Aligne le texte selon les constantes d'alignement Qt.AlignLeft Qt.AlignRight Qt.AlignCenter Qt.AlignJustify
2	**setIndent()** Définit le retrait du texte des étiquettes
3	**setPixmap()** Affiche une image
4	**text()** Affiche la légende de l'étiquette
5	**setText()** Définit la légende par programme
6	**selectedText()** Affiche le texte sélectionné à partir de l'étiquette(le textInteractionFlag doit être défini sur TextSelectableByMouse)
7	**setBuddy()** Associe l'étiquette à n'importe quel widget d'entrée
8	**setWordWrap()** Active ou désactive l'habillage du texte dans l'étiquette
9	**adjustSize()** Changera la taille de l'étiquette en fonction de la longueur du texte
10	**move(x,y)** Permet de positionner l'étiquette sur une position absolue

Signaux de la classe **QLabel**

linkActivated	Si vous cliquez sur l'étiquette contenant le lien hypertexte intégré, l'URL s'ouvrira. La fonction setOpenExternalLinks doit être définie sur True.
linkHovered	La méthode de slot associée à ce signal sera appelée lorsque l'étiquette ayant un lien hypertexte intégré est survolée par la souris.

QLineEdit Widget

L'objet **QLineEdit** est le champ de saisie le plus couramment utilisé. Il fournit une boîte dans laquelle une ligne de texte peut être saisie. Afin de saisir du texte **multiligne, l'objet QTextEdit** est requis.
Le tableau suivant répertorie quelques méthodes importantes de la classe **QLineEdit** :

N°	Méthodes et description

1	**setAlignment()** Aligne le texte selon les constantes d'alignement ➢ Qt.AlignLeft ➢ Qt.AlignRight ➢ Qt.AlignCenter ➢ Qt.AlignJustify
2	**clear()** Efface le contenu
3	**setEchoMode()** Contrôle l'apparence du texte à l'intérieur de la zone. Les valeurs d'échomode sont : ➢ QLineEdit.Normal ➢ QLineEdit.NoEcho ➢ QLineEdit.Password ➢ QLineEdit.PasswordEchoOnEdit
4	**setMaxLength()** Définit le nombre maximum de caractères pour la saisie
5	**setReadOnly()** Rend la zone de texte non modifiable
6	**setText()** Définit le texte par programmation
7	**text()** Récupère le texte dans le champ
8	**setValidator()** Définit les règles de validation. Les validateurs disponibles sont : ✓ **QIntValidator** : Restreint l'entrée à un entier ✓ **QDoubleValidator** : **Partie de** fraction du nombre limitée aux décimales spécifiées ✓ **QRegexpValidator** : Vérifie l'entrée par rapport à une expression Regex
9	**setInputMask()** Applique un masque de combinaison de caractères pour la saisie
10	**setFont()** Affiche le contenu de l'objet QFont
11	**hide()** Permet de cacher le widget

L'objet **QLineEdit** émet les signaux suivants :

N°	Méthodes et description
1	**cursorPositionChanged()** Chaque fois que le curseur bouge
2	**editingFinished()** Lorsque vous appuyez sur 'Entrée' ou que le champ perd le focus
3	**returnPressed()** Lorsque vous appuyez sur 'Entrée'
4	**selectionChanged()** Chaque fois que le texte sélectionné change
5	**textChanged()** Au fur et à mesure que le texte de la boîte change par saisie ou par programme
6	**textEdited()** Chaque fois que le texte est modifié

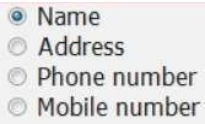
sys.exit(app.exec

Widget QPushButton

Dans toute conception d'interface graphique, le bouton de commande est le contrôle le plus important et le plus souvent utilisé. Les boutons avec Enregistrer, Ouvrir, OK, Oui, Non et Annuler, etc. comme légende sont familiers à tout utilisateur d'ordinateur. Dans l'API PyQt, l'objet de classe **QPushButton** présente un bouton qui, lorsqu'il est cliqué, peut être programmé pour appeler une certaine fonction.
La classe **QPushButton** hérite de sa fonctionnalité principale de la classe **QAbstractButton** . Il est de forme rectangulaire et une légende de texte ou une icône peut être affichée sur son visage.
Voici quelques-unes des méthodes les plus couramment utilisées de la classe **QPushButton** :

N°	Méthodes et description
1	**setCheckable()** Reconnaît les états enfoncés et relâchés du bouton s'il est défini sur True
2	**toggle()** Bascule entre les états vérifiables
3	**setIcon()** Affiche une icône formée à partir de la pixmap d'un fichier image
4	**setEnabled()** Lorsqu'il est défini sur False, le bouton devient désactivé, donc cliquer dessus n'émet pas de signal
5	**isChecked()** Renvoie l'état booléen du bouton
6	**setDefault()** Définit le bouton par défaut
7	**setText()** Définit par programmation la légende des boutons
8	**text()** Récupère la légende des boutons

Widget QRadioButton

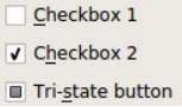


Un objet de classe **QRadioButton** présente un bouton sélectionnable avec une étiquette de texte. L'utilisateur peut sélectionner l'une des nombreuses options présentées sur le formulaire. Cette classe est dérivée de la classe QAbstractButton.
Les boutons radio sont auto-exclusifs par défaut. Par conséquent, un seul des boutons radio de la fenêtre parent peut être sélectionné à la fois. Si l'un d'entre eux est sélectionné, le bouton précédemment sélectionné est automatiquement désélectionné. Les boutons radio peuvent également être placés dans un **QGroupBox** ou un **QButtonGroup** pour créer plusieurs champs sélectionnables dans la fenêtre parente.
Les méthodes répertoriées suivantes de la classe **QRadioButton** sont les plus couramment utilisées :

N°	Méthodes et description
1	**setChecked()** Modifie l'état du bouton radio
2	**setText()** Définit le libellé associé au bouton
3	**text()** Récupère la légende du bouton
4	**isChecked()** Vérifie si le bouton est sélectionné

Le signal par défaut associé à l'objet QRadioButton est **toggled**(), bien que d'autres signaux hérités de la classe QAbstractButton puissent également être implémentés.

Widget QCheckBox



Une zone rectangulaire avant l'étiquette de texte apparaît lorsqu'un objet **QCheckBox** est ajouté à la fenêtre parente. Tout comme QRadioButton, il s'agit également d'un bouton sélectionnable. Son utilisation courante est dans un scénario où l'utilisateur est invité à choisir une ou plusieurs des options disponibles.
Contrairement aux boutons radio, les cases à cocher ne sont pas mutuellement exclusives par défaut. Afin de restreindre le choix à l'un des éléments disponibles, les cases à cocher doivent être ajoutées à QButtonGroup.
Le tableau suivant répertorie les méthodes de classe **QCheckBox** couramment utilisées :

N°	Méthodes et description
1	**setChecked()** Modifie l'état du bouton de la case à cocher
2	**setText()** Définit le libellé associé au bouton
3	**text()** Récupère la légende du bouton
4	**isChecked()** Vérifie si le bouton est sélectionné
5	**setTriState()** Ne fournit aucun état de changement à la case à cocher
6	**toggle()** Coche / décoche la case à cocher

Chaque fois qu'une case est cochée ou décochée, l'objet émet le signal **stateChanged**().

Widget QComboBox

Un objet **QComboBox** présente une liste déroulante d'éléments à sélectionner. Il faut un espace d'écran minimum sur le formulaire requis pour afficher uniquement l'élément actuellement sélectionné.
Une zone de liste déroulante peut être définie pour être modifiable; il peut également stocker des objets pixmap. Les méthodes suivantes sont couramment utilisées de QComboBox :

N°	Méthodes et description
1	**addItem()** Ajoute une chaîne à la collection
2	**addItems()** Ajoute des éléments dans un objet de liste
3	**clear()** Supprime tous les éléments de la collection
4	**count()** Récupère le nombre d'éléments de la collection
5	**currentText()** Récupère le texte de l'élément actuellement sélectionné
6	**itemText()** Affiche le texte appartenant à un index spécifique
7	**currentIndex()** Renvoie l'index de l'élément sélectionné
8	**setItemText()** Modifie le texte de l'index spécifié

Signaux ***QComboBox***

N°	Méthodes et description
1	**activated()** Lorsque l'utilisateur choisit un élément
2	**currentIndexChanged()** Chaque fois que l'index actuel est modifié par l'utilisateur ou par programme
3	**highlighted()** Lorsqu'un élément de la liste est mis en surbrillance

Widget QSpinBox

Un objet **QSpinBox** présente à l'utilisateur une zone de texte qui affiche un entier avec le bouton haut / bas à sa droite. La valeur dans la zone de texte augmente / diminue si le bouton haut / bas est enfoncé.
Par défaut, le nombre entier dans la zone commence par 0, va jusqu'à 99 et change à l'étape 1. Utilisez QDoubleSpinBox pour les valeurs flottantes.
Les méthodes importantes de la classe **QSpinBox** sont répertoriées dans le tableau suivant :

N°	Méthodes et description
1	**setMinimum()** Définit la limite inférieure du compteur
2	**setMaximum()** Définit la limite supérieure du compteur
3	**setRange()** Définit la valeur minimum, maximum et pas
4	**setValue()** Définit la valeur de la boîte de sélection numérique par programmation
5	**value()** Renvoie la valeur actuelle
6	**singleStep()** Définit la valeur de pas du compteur

L'objet **QSpinBox** émet le signal **valueChanged()** à chaque fois que vous appuyez sur le bouton haut / propre. La fonction slot associée peut récupérer la valeur actuelle du widget par la méthode **value()**.

QSlider

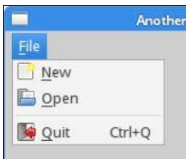
L'objet de classe **QSlider** présente à l'utilisateur une pluie sur laquelle une poignée peut être déplacée. C'est un widget classique pour contrôler une valeur bornée. La position de la poignée sur la pluie équivaut à un entier entre les limites inférieures et supérieure de la commande.
Un contrôle de curseur peut être affiché de manière horizontale ou verticale en mentionnant l'orientation dans le constructeur.

N°	Méthodes et description	
1	**setMinimum()** Définit la limite inférieure du curseur	
2	**setMaximum()** Définit la limite supérieure du curseur	
3	**setSingleStep()** Définit le pas d'incrémentation / décrémentation	
4	**setValue()** Définit la valeur du contrôle par programme	
5	**value()** Renvoie la valeur actuelle	
6	**setTickInterval()** Met le nombre de ticks sur la rainure	
7	**setTickPosition()** Place les graduations sur la rainure. Les valeurs sont :	
	QSlider.NoTicks	Pas de graduation
	QSlider.TicksBothSides	Coche des deux côtés
	QSlider.TicksAbove	Coche au-dessus du curseur
	QSlider.TicksBelow	Coche sous le curseur
	QSlider.TicksLeft	Coche à gauche du curseur
	QSlider.TicksRight	Coche à droite du curseur

Signaux **QSlider**

N°	Méthodes et description
1	**valueChanged()** Lorsque la valeur du curseur a changé
2	**sliderPressed()** Lorsque l'utilisateur commence à faire glisser le curseur
3	**sliderMoved()** Lorsque l'utilisateur fait glisser le curseur
4	**sliderReleased()** Lorsque l'utilisateur relâche le curseur

Widgets QMenuBar, QMenu et QAction



Une **QMenuBar** horizontale juste en dessous de la barre de titre d'un objet QMainWindow est réservée à l'affichage des objets QMenu.
La classe **QMenu** fournit un widget qui peut être ajouté à la barre de menus. Il est également utilisé pour créer un menu contextuel et un menu contextuel. Chaque objet QMenu peut contenir un ou plusieurs objets **QAction** ou des objets QMenu en cascade.
Pour créer un menu contextuel, l'API PyQt fournit la fonction createPopupMenu(). La fonction menuBar() renvoie l'objet QMenuBar de la fenêtre principale. La fonction addMenu() permet d'ajouter un menu à la barre. À leur tour, les actions sont ajoutées dans le menu par la méthode addAction().
Le tableau suivant répertorie certaines des méthodes importantes utilisées dans la conception d'un système de menus.
Vous trouverez ci-dessous les méthodes les plus couramment utilisées de **QMenu**.

N°	Méthodes et description
1	**menuBar()** Renvoie l'objet QMenuBar de la fenêtre principale
2	**addMenu()** Ajoute un nouvel objet QMenu à la barre de menus
3	**addAction()** Ajoute un bouton d'action au widget QMenu composé de texte ou d'icône
4	**setEnabled()** Définit l'état du bouton d'action sur activé / désactivé
5	**addSeperator()** Ajoute une ligne de séparation dans le menu
6	**clear()** Supprime le contenu du menu / barre de menu
7	**setShortcut()** Associe le raccourci clavier au bouton d'action
8	**setText()** Attribue du texte au bouton d'action
9	**setTitle()** Définit le titre du widget QMenu
10	**text()** Récupère le texte associé à l'objet QAction
11	**title()** Récupère le texte associé à l'objet QMenu

L'objet **QMenu** émet un signal **triggered**() chaque fois qu'un bouton **QAction** est cliqué. La référence à l'objet QAction cliqué est transmise à la fonction de slot connecté.

Widget QToolBar

Un widget **QToolBar** est un panneau mobile composé de boutons de texte, de boutons avec des icônes ou d'autres widgets. Il est généralement situé dans une barre horizontale sous la barre de menus, bien qu'il puisse être flottant. Voici quelques méthodes utiles de la classe **QToolBar**:

N°	Méthodes et description
1	**addAction()** Ajoute des boutons d'outils contenant du texte ou une icône
2	**addSeperator()** Affiche les boutons d'outils en groupes
3	**addWidget()** Ajoute des contrôles autres que le bouton dans la barre d'outils
4	**addToolBar()** La méthode de classe QMainWindow ajoute une nouvelle barre d'outils
5	**setMobile()** La barre d'outils devient mobile
6	**setOrientation()** L'orientation de la barre d'outils est définie sur Qt.horizontal ou Qt.vertical

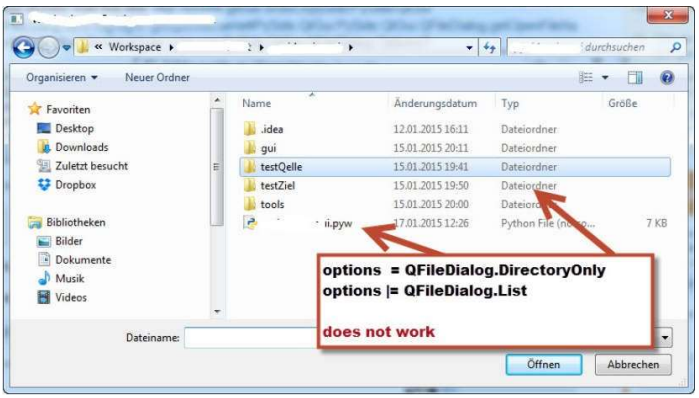
Chaque fois qu'un bouton de la barre d'outils est cliqué, le signal **ActionTriggered()** est émis. De plus, il envoie une référence à l'objet QAction associé à l'événement à la fonction connectée.

Widget QInputDialog

Il s'agit d'une boîte de dialogue préconfigurée avec un champ de texte et deux boutons, OK et Annuler. La fenêtre parente recueille l'entrée dans la zone de texte après que l'utilisateur clique sur le bouton OK ou appuie sur Entrée.
L'entrée utilisateur peut être un nombre, une chaîne ou un élément de la liste. Une étiquette demandant à l'utilisateur ce qu'il doit faire est également affichée.
La classe **QInputDialog** a les méthodes statiques suivantes pour accepter l'entrée de l'utilisateur :

N°	Méthodes et description
1	**getInt()** Crée une boîte de sélection pour un nombre entier
2	**getDouble()** La boîte de spinner avec un nombre à virgule flottante peut être entrée
3	**getText()** Un champ d'édition de ligne simple pour saisir du texte
4	**getItem()** Une zone de liste déroulante à partir de laquelle l'utilisateur peut choisir un élément

Widget QFileDialog



Ce widget est une boîte de dialogue de sélection de fichiers. Il permet à l'utilisateur de naviguer dans le système de fichiers et de sélectionner un fichier à ouvrir ou à enregistrer. Le dialogue est appelé via des fonctions statiques ou en appelant la fonction exec_() sur l'objet de dialogue.

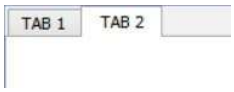
Les fonctions statiques de la classe **QFileDialog(**getOpenFileName() et getSaveFileName()) appellent la boîte de dialogue de fichier native du système d'exploitation actuel.

Un filtre de fichier peut également être appliqué pour n'afficher que les fichiers des extensions spécifiées. Le répertoire de départ et le nom de fichier par défaut peuvent également être définis.

Les méthodes et énumérations importantes de la classe **QFileDialog** sont répertoriées dans le tableau suivant :

N°	Méthodes et description
1	**getOpenFileName()** Renvoie le nom du fichier sélectionné par l'utilisateur pour l'ouvrir
2	**getSaveFileName()** Utilise le nom de fichier sélectionné par l'utilisateur pour enregistrer le fichier
3	**setacceptMode()** Détermine si la boîte de fichier agit comme une boîte de dialogue d'ouverture ou d'enregistrement ➢ QFileDialog.AcceptOpen ➢ QFileDialog.AcceptSave
4	**setFileMode()** Type de fichiers sélectionnables. Les constantes énumérées sont : ➢ QFileDialog.AnyFile ➢ QFileDialog.ExistingFile ➢ QFileDialog.Directory ➢ QFileDialog.ExistingFiles
5	**setFilter()** Affiche uniquement les fichiers ayant des extensions mentionnées

Widget QTab



Si un formulaire comporte trop de champs pour être affichés simultanément, ils peuvent être organisés en différentes pages placées sous chaque onglet d'un widget à onglets. Le **QTabWidget** fournit une barre d'onglets et une zone de page. La page sous le premier onglet s'affiche et les autres sont masquées. L'utilisateur peut afficher n'importe quelle page en cliquant sur l'onglet souhaité.

Voici quelques-unes des méthodes fréquemment utilisées de la classe **QTabWidget** :

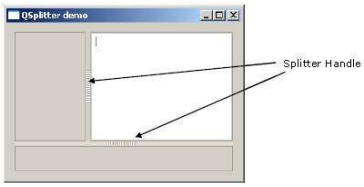
N°	Méthodes et description
1	**addTab()** Ajoute un onglet associé à une page de widget
2	**insertTab()** Insère un onglet avec la page à la position souhaitée
3	**removeTab()** Supprime l'onglet à un index donné
4	**setCurrentIndex()** Définit l'index de la page actuellement visible comme courant
5	**setCurrentWidget()** Rend la page visible comme actuelle
6	**setTabBar()** Définit la barre d'onglets du widget
7	**setTabPosition()**

	La position des onglets est contrôlée par les valeurs ➢ QTabWidget.Nord au-dessus des pages ➢ QTabWidget.South sous les pages ➢ QTabWidget.West à gauche des pages ➢ QTabWidget.East à droite des pages
8	**setTabText()** Définit l'étiquette associée à l'index de l'onglet

Les signaux suivants sont associés à l'objet **QTabWidget** :

N°	Méthodes et description
1	**currentChanged()** Chaque fois que l'index de la page actuelle change
2	**tabClosedRequested()** Lorsque vous cliquez sur le bouton de fermeture de l'onglet

Widget QSplitter



Il s'agit d'un autre gestionnaire de mise en page avancé qui permet de modifier dynamiquement la taille des widgets enfants en faisant glisser les limites entre eux. Le contrôle Splitter fournit une poignée qui peut être déplacée pour redimensionner les contrôles. Les widgets d'un objet **QSplitter** sont disposés horizontalement par défaut bien que l'orientation puisse être modifiée en Qt.Vertical.

Voici les méthodes et les signaux de la classe **QSplitter** :

N°	Méthodes et description
1	**addWidget()** Ajoute le widget à la disposition du séparateur
2	**indexOf()** Renvoie l'index du widget dans la mise en page
3	**insetWidget()** Insère un widget à l'index spécifié
4	**setOrientation()** Définit la disposition du séparateur sur Qt.Horizontal ou Qt.Vertical
5	**setSizes()** Définit la taille initiale de chaque widget
6	**count()** Renvoie le nombre de widgets dans le widget de fractionnement

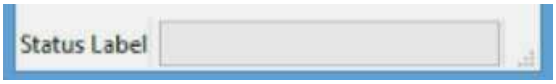
splitterMoved() est le seul signal émis par l'objet **QSplitter** chaque fois que la poignée du séparateur est déplacée.

Widget QDock

Une fenêtre ancrable est une sous-fenêtre qui peut rester à l'état flottant ou peut être attachée à la fenêtre principale à une position spécifiée. L'objet de fenêtre principale de la classe QMainWindow a une zone réservée aux fenêtres ancrables. Cette zone est autour du widget central.
Une fenêtre de dock peut être déplacée à l'intérieur de la fenêtre principale, ou elle peut être désancrée pour être déplacée dans une nouvelle zone par l'utilisateur. Ces propriétés sont contrôlées par les méthodes de classe **QDockWidget** suivantes :

N°	Méthodes et description
1	**setWidget()** Définit n'importe quel QWidget dans la zone de la fenêtre du dock
2	**setFlottant()** Si la valeur est true, la fenêtre du dock peut flotter
3	**setAllowedAreas()** Définit les zones auxquelles la fenêtre peut-être ancrée LeftDockWidgetArea RightDockWidgetArea TopDockWidgetArea BottomDockWidgetArea NoDockWidgetArea
4	**setFeatures()** Définit les fonctionnalités de la fenêtre du dock DockWidgetClosable DockWidgetMobile DockWidgetFloatable DockWidgetVerticalTitleBar NoDockWidgetCaractéristiques

Widget QStatusBar



L'objet **QMainWindow** réserve une barre horizontale en bas comme **barre d'état.** Il est utilisé pour afficher des informations d'état permanentes ou contextuelles.
Il existe trois types d'indicateurs d'état -

- **Temporaire** : occupe brièvement la majeure partie de la barre d'état. Par exemple, utilisé pour expliquer les textes des info-bulles ou les entrées de menu.
- **Normal** : occupe une partie de la barre d'état et peut être masqué par des messages temporaires. Par exemple, utilisé pour afficher la page et le numéro de ligne dans un traitement de texte.
- **Permanent** : Il n'est jamais caché. Utilisé pour les indications de mode importantes. Par exemple, certaines applications mettent un indicateur de verrouillage des majuscules dans la barre d'état.

Méthodes de la classe **QStatusBar :**

N°	Méthodes et description
1	**addWidget()** Ajoute l'objet widget donné dans la barre d'état
2	**addPermanentWidget()** Ajoute définitivement l'objet widget donné dans la barre d'état
3	**showMessage()** Affiche un message temporaire dans la barre d'état pendant un intervalle de temps spécifié
4	**clearMessage()** Supprime tout message temporaire affiché
5	**removeWidget()** Supprime le widget spécifié de la barre d'état

Widget QList

La classe **QListWidget** est une interface basée sur des éléments pour ajouter ou supprimer des éléments d'une liste. Chaque élément de la liste est un objet QListWidgetItem. ListWidget peut être défini pour être multi-sélectionnable.
Voici les méthodes fréquemment utilisées de la classe **QListWidget** :

N°	Méthodes et description
1	**addItem()** Ajoute un objet ou une chaîne QListWidgetItem dans la liste
2	**addItems()** Ajoute chaque élément de la liste
3	**insertItem()** Insère l'élément à l'index spécifié
4	**clear()** Supprime le contenu de la liste
5	**setCurrentItem()** Définit l'élément actuellement sélectionné par programme
6	**sortItems()** Réorganise les éléments par ordre croissant

Voici les signaux émis par **QListWidget** :

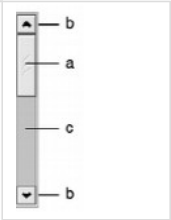
N°	Méthodes et description
1	**currentItemChanged()** Chaque fois que l'élément actuel change
2	**itemClicked()** Chaque fois qu'un élément de la liste est cliqué

Widget QScrollBar

Un contrôle de barre de défilement permet à l'utilisateur d'accéder à des parties du document qui se trouvent en dehors de la zone visible. Il fournit un indicateur visuel de la position actuelle. Il dispose d'un curseur par lequel une valeur entre une plage prédéfinie est définie de manière analogue. Cette valeur est généralement corrélée pour amener une donnée masquée dans la fenêtre.
Le contrôle de la barre de défilement a quatre contrôles :

Un curseur

Deux flèches de défilementContrôle de page



Les signaux suivants de la classe **QScrollBar** sont fréquemment utilisés :

N°	Méthodes et description
1	**valueChanged()** Lorsque la valeur de la barre de défilement change
2	**sliderMoved()** Lorsque l'utilisateur fait glisser le curseur

Widget QCalendar

Le widget **QCalendar** est un contrôle de sélection de date utile. Il fournit une vue mensuelle. L'utilisateur peut sélectionner la date à l'aide de la souris ou du clavier, la date par défaut étant la date du jour. La plage de dates du calendrier peut également être stipulée.

Vous trouverez ci-dessous les méthodes les plus couramment utilisées de **QCalendar :**

N°	Méthodes et description
1	**setDateRange()** Définit les dates inférieure et supérieure disponibles pour la sélection
2	**setFirstDayOfWeek()** Détermine le jour de la première colonne du calendrier Les constantes de jour prédéfinies sont - ➢ Qt.Monday ➢ Qt.Tuesday ➢ Qt.Wednesday ➢ Qt.Thursday ➢ Qt.Friday ➢ Qt.Saturday ➢ Qt.Sunday
3	**setMinimumDate()** Définit la date la plus basse pour la sélection
4	**setMaximumDate()** Définit la date supérieure de sélection
5	**setSelectedDate()** Définit un objet QDate comme date sélectionnée
6	**showToday()** Affiche le mois d'aujourd'hui
7	**selectedDate()** Récupère la date sélectionnée
8	**setGridvisible()** Active ou désactive la grille du calendrier

QLCDNumber

Le widget **QLCDNumber** affiche un nombre avec des chiffres de type LCD.

Il peut afficher un nombre dans à peu près n'importe quelle taille. Il peut afficher des nombres décimaux, hexadécimaux, octaux ou binaires. Il est facile de se connecter aux sources de données à l'aide du slot display (), qui est surchargé pour accepter l'un des cinq types d'argument.

Il existe également des emplacements pour changer la base avec setMode () et le point décimal avec setSmallDecimalPoint ().

QLCDNumber émet le signal de débordement () quand on lui demande d'afficher quelque chose au-delà de sa plage. La plage est définie par setDigitCount (), mais setSmallDecimalPoint () l'influe également. Si l'affichage est défini sur hexadécimal, octal ou binaire, l'équivalent entier de la valeur s'affiche.

Ces chiffres et autres symboles peuvent être affichés: 0 / O, 1, 2, 3, 4, 5 / S, 6, 7, 8, 9 / g, moins, point décimal, A, B, C, D, E, F, h, H, L, o, P, r, u, U, Y, deux-points, signe de degré (qui est spécifié comme guillemet simple dans la chaîne) et espace.

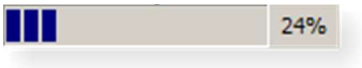
QLCDNumber remplace les caractères non autorisés par des espaces.

Il n'est pas possible de récupérer le contenu d'un objet QLCDNumber, bien que vous puissiez récupérer la valeur numérique avec value (). Si vous avez vraiment besoin du texte, nous vous recommandons de connecter également les signaux qui alimentent le slot display () à un autre slot et d'y stocker la valeur.

Les méthodes et énumérations importantes associées à la classe **QLCDNumber** sont données dans le tableau suivant :

N°	Méthodes et description
1	**display()** Affiche la valeur donnée comme paramètre
2	**value()** Renvoie la valeur actuelle

QProgressBar



Une barre de progression permet de donner à l'utilisateur une indication de la progression d'une opération et de le rassurer que l'application est toujours en cours d'exécution.
La barre de progression utilise le concept d'étapes. Vous le configurez en spécifiant les valeurs d'étape minimum et maximum possibles, et il affichera le pourcentage d'étapes qui ont été achevées lorsque vous lui donnerez ultérieurement la valeur d'étape actuelle. Le pourcentage est calculé en divisant la progression (valeur () - minimum ()) divisée par maximum () - minimum ().
Vous pouvez spécifier le nombre minimum et maximum d'étapes avec setMinimum () et setMaximum. Le nombre actuel de pas est défini avec setValue (). La barre de progression peut être rembobinée au début avec reset ().
Si le minimum et le maximum sont tous deux définis sur 0, la barre affiche un indicateur d'occupation au lieu d'un pourcentage d'étapes. Ceci est utile, par exemple, lors de l'utilisation de QNetworkAccessManager pour télécharger des éléments lorsqu'ils ne peuvent pas déterminer la taille de l'élément en cours de téléchargement.
Les méthodes et énumérations importantes associées à la classe **QProgressBar** sont données dans le tableau suivant :

N°	Méthodes et description
1	**setValue()** Fixe la valeur de la barre de progression
2	**value()** Renvoie la valeur actuelle

QClipboard

La classe **QClipboard** donne accès au presse-papiers à l'échelle du système qui offre un mécanisme simple pour copier et coller des données entre les applications. Son action est similaire à la classe QDrag et utilise des types de données similaires.
La classe **QApplication** a une méthode statique **clipboard()** qui renvoie une référence à un objet clipboard. Tout type de MimeData peut être copié ou collé à partir du presse-papiers.
Voici les méthodes de classe de presse-papiers qui sont couramment utilisées :

N°	Méthodes et description
1	**clear()** Efface le contenu du presse-papiers
2	**setImage()** Copie QImage dans le presse-papiers
3	**setMimeData()** Définit les données MIME dans le presse-papiers
4	**setPixmap()** Copie l'objet Pixmap dans le presse-papiers
5	**setText()** Copie QString dans le presse-papiers
6	**text()** Récupère le texte du presse-papiers

Le signal associé à l'objet presse-papiers est :

N°	Méthodes et description
1	**dataChanged()** Chaque fois que les données du presse-papiers changent

QMessageBox

QMessageBox est une boîte de dialogue modale couramment utilisée pour afficher un message d'information et éventuellement demander à l'utilisateur de répondre en cliquant sur l'un des boutons standard. Chaque bouton standard a une légende prédéfinie, un rôle et renvoie un nombre hexadécimal prédéfini.
Les méthodes et énumérations importantes associées à la classe **QMessageBox** sont données dans le tableau suivant :

N°	Méthodes et description
1	**setIcon()** Affiche l'icône prédéfinie correspondant à la gravité du message Question — Warning Information — Critical
2	**setText()** Définit le texte du message principal à afficher
3	**setInformativeText()** Affiche des informations supplémentaires
4	**setDetailText()** La boîte de dialogue affiche un bouton Détails. Ce texte apparaît en cliquant dessus
5	**setTitle()** Affiche le titre personnalisé de la boîte de dialogue
6	**setStandardButtons()** Liste des boutons standard à afficher. Chaque bouton est associé à ➢ QMessageBox.Ok 0x00000400 ➢ QMessageBox.Open 0x00002000 ➢ QMessageBox.Save 0x00000800 ➢ QMessageBox.Cancel 0x00400000 ➢ QMessageBox.Close 0x00200000 ➢ QMessageBox.Yes 0x00004000 ➢ QMessageBox.No 0x00010000 ➢ QMessageBox.Abort 0x00040000 ➢ QMessageBox.Retry 0x00080000 ➢ QMessageBox.Ignore 0x00100000